



Healthcare Predictive Analytics / Stroke Risk Prediction

A Comprehensive Data Science Project Proposal

Project Team

The following team members are all designated as Data Scientists:

Muuhmd Salah Abd El-Bari (Team Leader)

Nermine Samir Abd El-Aty

Toqa Hamed Mohammed

George Emil Sadik

Mostafa Mohammed Fouad

Omar Ahmed Mostafa

AI & Data Science Track - Round 2

Abstract

This document presents a complete project proposal for a Healthcare Predictive Analytics system focused on stroke risk prediction. The project aims to analyze patient demographic, clinical, and lifestyle indicators, identify major patterns associated with stroke risk, and build a machine learning model that estimates the probability of stroke occurrence. The proposed solution follows a full data science lifecycle: project planning, dataset understanding, data preprocessing, exploratory analysis, feature engineering, model development, experiment tracking, deployment, monitoring, and final documentation.

The project uses a structured healthcare dataset containing features such as age, gender, hypertension status, heart disease status, marital status, work type, residence type, average glucose level, body mass index, smoking status, and stroke outcome. The system will compare multiple classification algorithms, including Logistic Regression, Random Forest, Gradient Boosting, and other suitable models, while handling the expected class imbalance in healthcare outcome data.

The final deliverable will include reproducible Jupyter notebooks, a trained model artifact, an interactive Streamlit application for real-time risk estimation, monitoring documentation, and a stakeholder-ready final report and presentation. The model is intended to support data-driven healthcare decision-making and educational analytics. It is not intended to replace clinical diagnosis or professional medical judgment.

Contents

Abstract	2
Contents	3
1 Project Planning & Management	5
1.1 Project Proposal	5
1.1.1 Overview	5
1.1.2 Project Objectives	5
1.1.3 Project Scope and Boundaries	5
1.2 Project Plan and Timeline	6
1.2.1 Gantt Chart - Project Timeline	6
1.2.2 Detailed Phase Breakdown and Deliverables	7
1.3 Task Assignment and Team Roles	7
1.4 Risk Assessment and Mitigation Plan	7
1.5 Key Performance Indicators (KPIs)	8
1.5.1 Technical KPIs	8
1.5.2 Healthcare and Analytical KPIs	9
2 Literature Review	10
2.1 Literature Review and Related Work	10
2.2 Academic Foundations	10
2.2.1 Healthcare Risk Factors and Predictive Modeling	10
2.2.2 Class Imbalance in Healthcare Outcomes	10
2.2.3 Explainability and Responsible AI	10
2.3 Dataset and Domain Context	10
2.4 Existing Data Science Project Patterns	11
2.5 Synthesis and Research Gap Identification	11
2.6 Methodological Framework Selection	11
3 Requirements Gathering	12

3.1 Stakeholder Analysis	12
3.2 User Stories	12
3.2.1 For Healthcare Professional Users	12
3.2.2 For Data Scientist Users	12
3.2.3 For Project Evaluator Users	12
3.2.4 For System Administrator Users	13
3.3 Use Case Diagram and Descriptions	13
3.3.1 Detailed Use Case Descriptions	13
3.4 Functional Requirements	14
3.5 Non-functional Requirements	15
4 System Analysis and Design	16
4.1 Problem Statement and Software Architecture	16
4.1.1 Problem Statement	16
4.1.2 Solution Approach	16
4.1.3 Software Architecture: Layered Design	16
4.2 Database Design and Data Modeling	16
4.2.1 Conceptual Data Model - Entity Relationship Overview	17
4.2.2 Logical Database Schema	17
4.2.3 Data Flow Diagram - Level 0 Context Diagram	18
4.2.4 Wireframe - Dashboard Main View	18
4.2.5 UI/UX Design Guidelines	19
4.3 Missing Diagrams - Recommendations and Resources	19
5 Expected Final Deliverables	20
Conclusion	20
References	21

1 Project Planning & Management

1.1 Project Proposal

1.1.1 Overview

Stroke is a critical healthcare outcome that benefits from early risk identification and proactive intervention. In many healthcare settings, clinical decision-making depends on multiple patient attributes that are difficult to evaluate consistently at scale. A data-driven predictive model can help healthcare professionals and analysts identify high-risk patient profiles, understand the key factors associated with stroke occurrence, and support resource planning.

This project proposes an end-to-end Healthcare Predictive Analytics pipeline for stroke risk prediction. The proposed system will:

- load and validate the healthcare stroke dataset;
- clean missing values, remove duplicates, handle inconsistent categories, and detect outliers;
- conduct exploratory data analysis to identify demographic, clinical, and lifestyle patterns;
- engineer model-ready features through encoding, scaling, grouping, and interaction features;
- train and compare multiple supervised classification models;
- optimize the final model using cross-validation and hyperparameter tuning;
- deploy the selected model in a Streamlit web application for real-time predictions;
- document monitoring, retraining, and responsible-use procedures.

The proposed dashboard will allow users to enter patient characteristics and receive an estimated stroke risk score with clear explanatory outputs. The application will emphasize interpretability, model limitations, and responsible use.

1.1.2 Project Objectives

Objective Type	Description	Success Metric
Primary Objective 1	Build a clean, reproducible healthcare data pipeline for stroke risk analytics.	100% of dataset columns documented; missing values and inconsistent categories handled through explicit preprocessing rules.
Primary Objective 2	Perform comprehensive exploratory data analysis on clinical and demographic variables.	At least 8 visualizations and 5 key analytical insights covering age, glucose, BMI, hypertension, heart disease, smoking, and stroke outcome.
Primary Objective 3	Develop and compare machine learning classification models for stroke risk prediction.	Model comparison using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix.
Primary Objective 4	Optimize the selected model for healthcare risk detection rather than accuracy alone.	Target ROC-AUC ≥ 0.80 and recall-oriented threshold selection to reduce false negatives.
Secondary Objective 1	Deploy the final model as an interactive Streamlit application.	Application accepts patient inputs, returns risk prediction, shows probability score, and explains major model drivers.
Secondary Objective 2	Implement basic MLOps and monitoring documentation.	Model artifact, preprocessing pipeline, experiment logs, versioning notes, drift-monitoring plan, and retraining strategy delivered.

Table 1: Project Objectives and Success Metrics

1.1.3 Project Scope and Boundaries

In-Scope Deliverables:

- Healthcare stroke dataset inspection, validation, and data dictionary creation.
- Data cleaning for missing values, duplicates, outliers, and inconsistent categories.
- Exploratory data analysis using histograms, boxplots, bar charts, correlation heatmaps, and class-balance plots.
- Feature engineering including categorical encoding, numeric scaling, risk grouping, and clinically meaningful interaction features.
- Supervised classification modeling for binary stroke prediction.
- Class imbalance treatment using stratified splitting, class weights, resampling, or threshold tuning.
- Model comparison using healthcare-relevant metrics such as recall, precision, F1-score, ROC-AUC, and confusion matrix.
- Streamlit web application for real-time patient risk estimation.
- Model monitoring plan covering data drift, prediction distribution drift, input validation, and retraining triggers.
- Final report, final presentation, notebooks, source code, and deployment instructions.

Out-of-Scope Exclusions:

- Direct clinical diagnosis, medical treatment recommendation, or replacement of physician judgment.
- Collection of new patient data from hospitals or clinics.
- Integration with electronic health record systems.
- Processing of personally identifiable information or protected health records.
- Regulatory approval as a medical device.
- Mobile application development; the deployment target is a web-based Streamlit dashboard.
- Real-time hospital infrastructure integration or production-grade cloud scaling beyond project demonstration.

1.2 Project Plan and Timeline

The project is organized into six phases over a 14-week implementation window. The schedule follows a data science lifecycle from planning and requirements through deployment and presentation.

1.2.1 Gantt Chart - Project Timeline

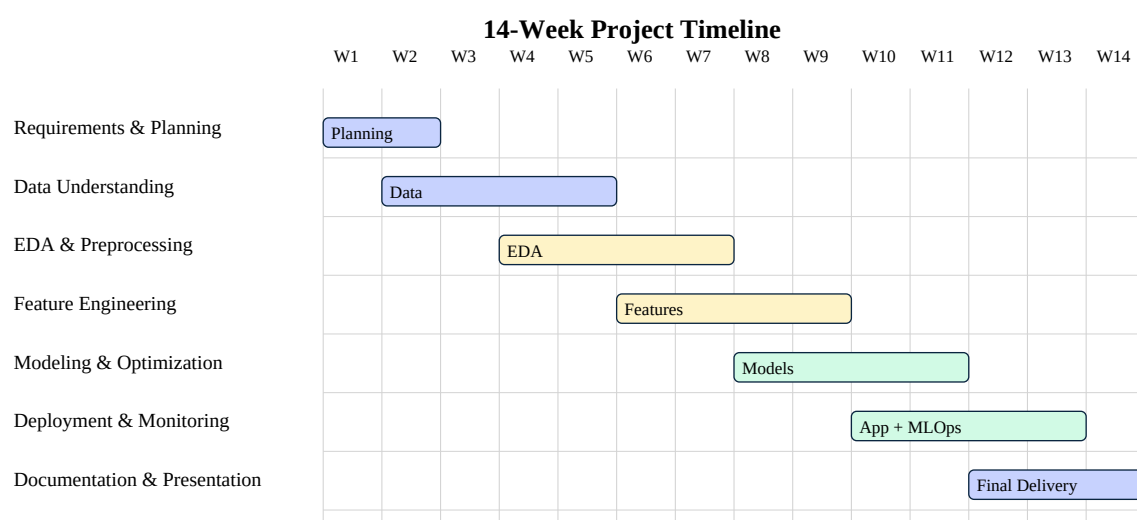


Figure 1: 14-Week Project Timeline for Stroke Risk Prediction

1.2.2 Detailed Phase Breakdown and Deliverables

Phase	Duration	Key Deliverables	Resources
Phase 1: Requirements & Planning	Weeks 1-2	Project proposal, scope definition, stakeholder analysis, user stories, initial risk matrix, success metrics.	Team Leader, all Data Scientists
Phase 2: Data Acquisition & Understanding	Weeks 2-5	Dataset loading, schema inspection, data dictionary, quality profile, missing-value report, initial notebook.	Data Scientists assigned to data understanding
Phase 3: EDA & Preprocessing	Weeks 5-7	Cleaned dataset, EDA notebook, visualizations, outlier analysis, preprocessing pipeline.	EDA and preprocessing team
Phase 4: Feature Engineering & Statistical Analysis	Weeks 6-9	Feature engineering notebook, hypothesis tests, feature selection summary, class imbalance treatment.	Feature engineering team
Phase 5: Modeling & Optimization	Weeks 8-11	Baseline models, tuned models, cross-validation results, evaluation report, selected final model.	Modeling and evaluation team
Phase 6: Deployment, Monitoring & Final Documentation	Weeks 10-14	Streamlit app, model artifact, requirements file, monitoring plan, final report, final presentation, demo script.	Deployment, MLOps, and documentation team

Table 2: Project Phases, Deliverables, and Resource Allocation

1.3 Task Assignment and Team Roles

All listed team members are designated as Data Scientists. To ensure full coverage of the project lifecycle, each member will own a primary responsibility while participating in shared review and integration sessions.

Role(s)	Team Member	Primary Responsibilities	Skills Required
Team Leader / ML Project Coordinator	Muuhmd Salah Abd El-Bari	Coordinate milestones, manage timeline, consolidate notebooks, review final model selection, supervise deployment readiness, maintain final deliverable checklist.	Project coordination, Python, ML lifecycle, evaluation metrics
Data Scientist - EDA Lead	Nermine Samir Abd El-Aty	Dataset profiling, missing-value analysis, descriptive statistics, exploratory plots, data quality summary, EDA notebook documentation.	Pandas, visualization, healthcare analytics, data cleaning
Data Scientist - Feature Engineering Lead	Toqa Hamed Mohammed	Encoding strategy, scaling strategy, feature creation, interaction variables, statistical tests, feature selection summary.	Scikit-learn preprocessing, statistics, feature selection
Data Scientist - Modeling Lead	George Emil Sadik	Train baseline and advanced classifiers, conduct cross-validation, tune hyperparameters, compare models, generate confusion matrices and ROC curves.	ML modeling, class imbalance methods, model evaluation
Data Scientist - Deployment Lead	Mostafa Mohammed Fouad	Package final model and preprocessing pipeline, build Streamlit app, create local execution instructions, verify app inputs and predictions.	Streamlit, Python packaging, model serialization, UI testing
Data Scientist - Documentation & QA Lead	Omar Ahmed Mostafa	Prepare final report and presentation, validate notebooks, check reproducibility, write monitoring and retraining documentation, support demo script.	Technical writing, QA, reproducibility, MLOps documentation

Table 3: Team Roles and Responsibilities

1.4 Risk Assessment and Mitigation Plan

Risk Category	Risk Description	Probability	Impact	Mitigation Strategy
Data Quality	Missing BMI values, inconsistent smoking categories, incorrect data types, or duplicate patient records.	High	High	Use explicit imputation rules, category normalization, duplicate checks, and a documented data quality report.
Class Imbalance	Stroke cases are expected to be much fewer than non-stroke cases, making accuracy misleading.	High	High	Use stratified splitting, class weights, resampling, PR/ROC analysis, recall-oriented threshold tuning, and confusion matrices.
Model Bias	Model may perform differently across age, gender, residence type, or smoking groups.	Medium	High	Evaluate subgroup performance, inspect feature importance, document limitations, and avoid unsupported clinical claims.
Clinical Misuse	Users may interpret the app as a medical diagnosis tool.	Medium	Very High	Add clear disclaimers, educational language, and guidance that outputs support analysis only and do not replace clinical judgment.
Overfitting	Complex models may memorize patterns and fail on unseen cases.	Medium	High	Use cross-validation, holdout testing, regularization, simple baselines, and model comparison.
Interpretability	Healthcare stakeholders may not trust black-box predictions.	Medium	Medium	Provide feature importance, model explanation plots, simple input summaries, and transparent methodology.
Deployment Failure	The Streamlit app may fail because of missing dependencies, path issues, or model artifact mismatch.	Medium	Medium	Deliver requirements.txt, relative paths, serialized pipeline, setup instructions, and end-to-end testing.
Monitoring Gap	Model performance may degrade with new populations or different input distributions.	Medium	High	Define drift checks, prediction distribution monitoring, periodic evaluation, and retraining triggers.
Timeline Risk	Scope may expand beyond proposal, delaying final delivery.	Medium	Medium	Use milestone-based planning, prioritize core deliverables, and reserve buffer time for QA.
Privacy and Ethics	Healthcare data may raise confidentiality concerns if real patient identifiers are introduced later.	Low	Very High	Use only anonymized project dataset, avoid PII, document ethical boundaries, and recommend governance before real-world adoption.

Table 4: Risk Assessment Matrix

1.5 Key Performance Indicators (KPIs)

The project KPIs are divided into technical KPIs and healthcare analytical KPIs.

1.5.1 Technical KPIs

KPI Name	Target Value	Measurement Method	Frequency
Notebook Reproducibility	All notebooks run top-to-bottom without manual path edits.	Restart kernel and run all cells.	At each milestone
Model Pipeline Integrity	Preprocessing and model inference saved as a single reusable pipeline.	Artifact load test and sample prediction test.	Before deployment
Streamlit App Startup Time	App starts locally within 30 seconds.	Local execution using Streamlit command.	Release testing
Model Inference Time	Prediction returned within 1 second for a single input.	Timing inside app or test script.	Weekly after deployment stage
Input Validation Coverage	All user inputs constrained by valid ranges/categories.	Manual and automated UI test cases.	Before final delivery
Documentation Completeness	README, notebooks, report, presentation, and app guide included.	Final deliverable checklist.	Final week

Table 5: Technical Performance KPIs

1.5.2 Healthcare and Analytical KPIs

KPI Name	Target Value	Measurement Method	Target Deadline
Data Completeness	Missing-value strategy defined for every feature.	Data quality report and preprocessing notebook.	End of Phase 3
EDA Insight Count	At least 5 major healthcare risk insights.	EDA report review.	End of Phase 3
Classification ROC-AUC	Target ≥ 0.80 on test data.	Holdout evaluation.	End of Phase 5
Stroke-Class Recall	Prioritize high recall for positive stroke cases.	Confusion matrix and classification report.	End of Phase 5
False Negative Review	Document false negative implications.	Model evaluation report.	End of Phase 5
Explainability Coverage	Provide feature importance or equivalent explanation.	Evaluation notebook and final report.	End of Phase 5
Dashboard Usability	Prediction form usable by non-technical stakeholders.	Demo walkthrough and peer review.	End of Phase 6

Table 6: Healthcare and Analytical KPIs

2 Literature Review

2.1 Literature Review and Related Work

The literature review focuses on healthcare risk prediction, machine learning classification, imbalanced data handling, and explainable analytics. Stroke risk prediction is a binary classification problem in which the positive outcome is clinically important but usually less frequent than the negative outcome. This makes the project different from standard balanced classification problems because accuracy alone can hide weak detection of actual high-risk cases.

Healthcare predictive analytics projects usually require a combination of statistical understanding and practical machine learning. The model must be evaluated not only on aggregate performance but also on the cost of errors. In this project, false negatives are especially important because a patient with elevated risk may be incorrectly classified as low risk. Therefore, recall, precision, F1-score, ROC-AUC, threshold tuning, and confusion matrix interpretation will be central to model selection.

2.2 Academic Foundations

2.2.1 Healthcare Risk Factors and Predictive Modeling

Stroke risk prediction depends on multiple patient characteristics. The dataset contains demographic variables such as age and gender; clinical variables such as hypertension, heart disease, average glucose level, and BMI; and lifestyle-related variables such as smoking status. Machine learning models can learn relationships between these variables and the target outcome, including non-linear patterns and interaction effects.

A simple baseline such as Logistic Regression is useful for interpretability and comparison. Tree-based methods such as Random Forest and Gradient Boosting can capture non-linear feature interactions and are often strong candidates for tabular healthcare prediction tasks. However, these models must be evaluated carefully to avoid overfitting and to ensure they provide reliable predictions on unseen patient profiles.

2.2.2 Class Imbalance in Healthcare Outcomes

Healthcare datasets often contain imbalanced outcomes because serious events are less common than normal cases. In stroke prediction, this means the number of patients with stroke can be much smaller than the number of patients without stroke. If this imbalance is ignored, a model may achieve high accuracy by predicting most cases as non-stroke while failing to detect the minority class.

To address this issue, the project will use stratified train-test splitting, class-weighted learning, threshold optimization, and potentially resampling methods. Evaluation will prioritize recall and ROC-AUC in addition to accuracy.

2.2.3 Explainability and Responsible AI

Healthcare stakeholders require transparent and interpretable outputs. A prediction without explanation may be difficult to trust or apply. The final system will therefore provide model explanations using feature importance, risk factor summaries, and clear descriptions of model limitations. The dashboard will avoid presenting model output as a clinical diagnosis and will communicate that the application is a decision-support and educational analytics tool.

2.3 Dataset and Domain Context

The project dataset is a structured healthcare stroke dataset. The main target variable is `stroke`, which indicates whether stroke occurred. Input features include `gender`, `age`, `hypertension`, `heart_disease`, `ever_married`, `work_type`, `Residence_type`, `avg_glucose_level`, `bmi`, `smoking_status`, and `stroke`.

The dataset is suitable for a supervised binary classification project. It also supports meaningful exploratory analysis, such as comparing stroke rates across age groups, glucose ranges, BMI categories, hypertension status, heart disease status, and smoking behavior.

2.4 Existing Data Science Project Patterns

- import and inspect the dataset;
- identify missing values and outliers;
- visualize distributions and feature-target relationships;
- encode categorical features and scale numerical variables;
- split the data into training and testing sets;
- train several classification models;
- compare performance using classification metrics;
- select and tune the best model;
- deploy the model through a web interface;
- document monitoring and retraining requirements.

2.5 Synthesis and Research Gap Identification

- Imbalance Gap: Many basic classification projects report high accuracy without adequately evaluating stroke-class recall.
- Explainability Gap: Healthcare users need interpretable risk factor summaries, not only model output labels.
- Deployment Gap: Many notebooks end at model evaluation and do not provide an interactive prediction interface.
- Monitoring Gap: Student projects often skip drift monitoring and retraining plans, even though healthcare data distributions can change.
- End-to-End Integration Gap: The project will connect data preprocessing, modeling, artifact management, deployment, and documentation into one reproducible workflow.

2.6 Methodological Framework Selection

- Data Collection: Load the provided healthcare stroke dataset and preserve raw data before transformations.
- Data Preprocessing: Handle missing BMI values, normalize categorical labels, remove duplicates, detect outliers, encode categorical variables, and scale numerical features.
- EDA: Generate histograms, bar charts, boxplots, correlation heatmaps, class balance plots, and feature-target comparisons.
- Feature Engineering: Create age groups, BMI categories, glucose-risk grouping, and clinical interaction features where appropriate.
- Modeling: Compare Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and other suitable classifiers.
- Optimization: Use cross-validation, class weights, threshold tuning, and hyperparameter search.
- Deployment: Package the model and preprocessing pipeline, then expose predictions through a Streamlit application.
- MLOps: Track experiments, store artifacts, document model versions, and define monitoring and retraining rules.

3 Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder Group	Role and Interest	Key Requirements	Engagement Strategy
Project Evaluators	Academic supervisors assessing deliverables against project requirements.	Complete documentation, reproducible notebooks, working Streamlit app, clear model comparison, final presentation.	Milestone reports, demo walkthrough, final project package.
Healthcare Professionals	Potential users interested in risk assessment and patient screening support.	Simple input form, interpretable output, clear limitations, risk factor explanation.	User-centered app demo, plain-language explanations, disclaimer.
Data Science Team	Team responsible for building and validating the analytics pipeline.	Clean data, reproducible code, versioned model artifacts, documented metrics.	Weekly integration sessions and shared notebooks.
Healthcare Administrators	Stakeholders interested in resource planning and high-risk patient segmentation.	Aggregated insights, risk-group distributions, monitoring reports, model performance summaries.	Dashboard summary, final report, KPI section.
Patients / Public Users	Indirect users who may benefit from educational risk-awareness tools.	Clear language, privacy protection, no misleading diagnosis claims.	Educational interface and responsible-use notes.
System Administrator	Person maintaining the app during demonstration or deployment.	Requirements file, logs, artifact paths, setup guide, monitoring plan.	README, deployment instructions, health-check documentation.

Table 7: Stakeholder Analysis Matrix

3.2 User Stories

User stories are expressed in the format: "As a user, I want a goal so that I can achieve a benefit."

3.2.1 For Healthcare Professional Users

- As a healthcare professional, I want to enter patient attributes so that I can receive a preliminary stroke risk estimate.
- As a healthcare professional, I want to see which factors influenced the prediction so that I can interpret the risk score responsibly.
- As a healthcare professional, I want warnings about model limitations so that I do not misuse the prediction as a diagnosis.
- As a healthcare professional, I want inputs to be validated so that impossible or invalid patient values are not accepted.

3.2.2 For Data Scientist Users

- As a data scientist, I want to run the preprocessing notebook from start to finish so that I can reproduce the cleaned dataset.
- As a data scientist, I want to compare several models using the same train-test split so that the comparison is fair.
- As a data scientist, I want to view feature importance so that I can understand model behavior.
- As a data scientist, I want to save the final pipeline artifact so that deployment uses the same transformations as training.

3.2.3 For Project Evaluator Users

- As a project evaluator, I want a clear final report so that I can verify the methodology and results.
- As a project evaluator, I want a working Streamlit demo so that I can test the deployed model.

- As a project evaluator, I want notebooks with visible outputs so that I can track each stage of the pipeline.
- As a project evaluator, I want task assignments so that I can evaluate individual team responsibilities.

3.2.4 For System Administrator Users

- As a system administrator, I want an installation guide so that I can run the application locally.
- As a system administrator, I want model and data file paths to be configurable so that the app can be moved between environments.
- As a system administrator, I want prediction logs so that I can monitor usage and detect drift signals.

3.3 Use Case Diagram and Descriptions

The use case diagram below illustrates the main interactions between system actors and the Healthcare Stroke Risk Prediction System.

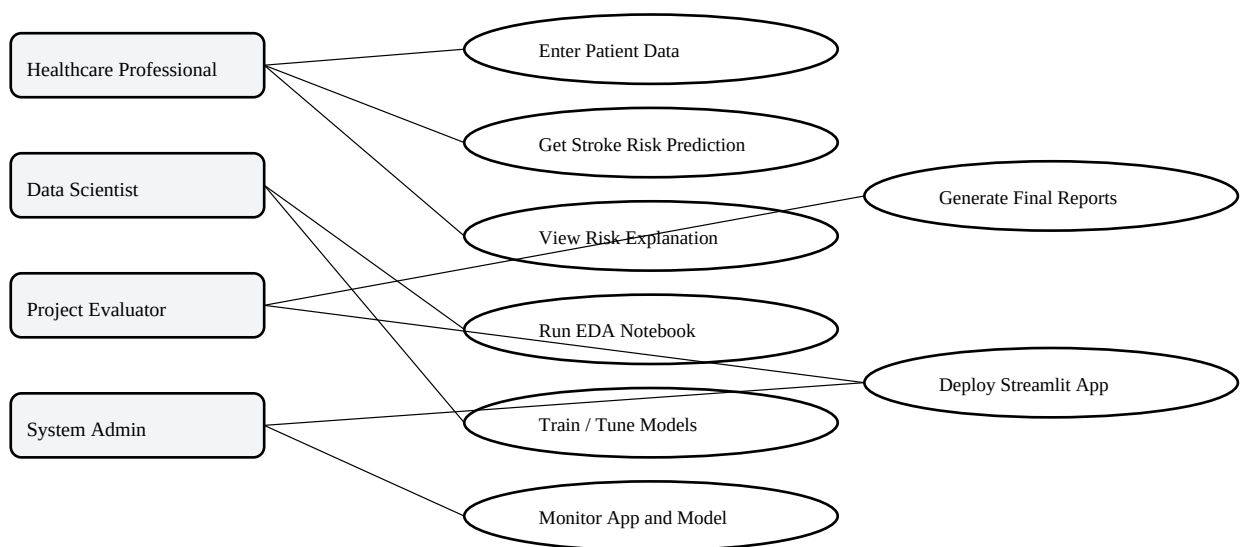


Figure 2: Use Case Diagram for Healthcare Stroke Risk Prediction System

3.3.1 Detailed Use Case Descriptions

UC-1: Enter Patient Data (Priority: High)

- Actor(s): Healthcare Professional, Project Evaluator
- Precondition: Streamlit app is running.
- Basic Flow: User enters age, gender, hypertension, heart disease, glucose level, BMI, smoking status, and related attributes.
- Alternative Flow: Invalid values trigger validation message.
- Postcondition: Inputs are transformed into the model-ready schema.

UC-2: Get Stroke Risk Prediction (Priority: High)

- Actor(s): Healthcare Professional, Project Evaluator
- Precondition: Final model and preprocessing pipeline are loaded.
- Basic Flow: User clicks predict; system returns predicted class and risk probability.
- Alternative Flow: Model artifact missing; app displays setup error.

- Postcondition: Prediction is shown with disclaimer and explanation.

UC-3: Train / Tune Models (Priority: High)

- Actor(s): Data Scientist
- Precondition: Cleaned data and preprocessing pipeline exist.
- Basic Flow: User trains multiple classifiers, performs cross-validation, tunes parameters, and stores results.
- Alternative Flow: Training fails due to missing dependencies or invalid data; error is logged and fixed.
- Postcondition: Best model is selected and exported.

UC-4: Monitor App and Model (Priority: Medium)

- Actor(s): System Administrator, Data Scientist
- Precondition: App is deployed or running locally.
- Basic Flow: System logs prediction requests, input distributions, and prediction probabilities.
- Alternative Flow: Drift threshold exceeded; retraining review is triggered.
- Postcondition: Monitoring report is available for review.

3.4 Functional Requirements

ID	Domain	Requirement Description	Priority
FR-01	Data Loading	System shall load the healthcare stroke dataset from the raw data directory.	High
FR-02	Data Quality	System shall identify missing values, duplicates, and invalid categories.	High
FR-03	Data Quality	System shall impute missing BMI values using a documented strategy.	High
FR-04	Preprocessing	System shall encode categorical variables using reusable transformers.	High
FR-05	Preprocessing	System shall scale numerical variables when required by the selected model.	Medium
FR-06	Analysis	System shall calculate descriptive statistics by stroke outcome.	High
FR-07	Visualization	System shall generate histograms, boxplots, bar charts, and heatmaps.	High
FR-08	Feature Engineering	System shall create clinically meaningful grouped features such as age group and glucose category where useful.	Medium
FR-09	Modeling	System shall train at least three classification algorithms.	High
FR-10	Modeling	System shall use stratified train-test splitting to preserve class distribution.	High
FR-11	Modeling	System shall evaluate models using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix.	High
FR-12	Modeling	System shall support hyperparameter tuning using grid search or randomized search.	Medium
FR-13	Modeling	System shall store the selected model and preprocessing pipeline as serialized artifacts.	High
FR-14	Explainability	System shall provide feature importance or equivalent explanation for the selected model.	Medium
FR-15	Dashboard	Streamlit app shall provide input widgets for all model features.	High
FR-16	Dashboard	Streamlit app shall output predicted class, probability, and risk interpretation.	High
FR-17	Dashboard	Streamlit app shall include responsible-use disclaimer.	High
FR-18	Monitoring	System shall define prediction logging and drift monitoring fields.	Medium
FR-19	Documentation	System shall include README, notebooks, final report, presentation, and deployment guide.	High
FR-20	Export	System shall save cleaned dataset and model evaluation outputs for reporting.	Medium

Table 8: Functional Requirements List

3.5 Non-functional Requirements

Category	ID	Requirement Specification
Performance	NFR-P01	Streamlit app initial load time should be less than 10 seconds on a standard laptop.
Performance	NFR-P02	Single prediction inference time should be less than 1 second.
Performance	NFR-P03	Notebooks should execute end-to-end within a reasonable project demonstration time.
Availability	NFR-A01	The local demo application should run using a documented setup command.
Security	NFR-S01	The project should not store personally identifiable information.
Security	NFR-S02	If future data is added, it must be anonymized before model training.
Usability	NFR-U01	Dashboard layout should use clear labels, tooltips, and validation messages.
Usability	NFR-U02	Prediction results should use plain language and avoid clinical overstatement.
Data Quality	NFR-DQ01	Data preprocessing choices must be documented in the notebook and final report.
Data Quality	NFR-DQ02	Model input schema must be consistent between training and deployment.
Maintainability	NFR-M01	Code should be organized into notebooks, scripts, app folder, model artifacts, and reports.
Maintainability	NFR-M02	Python files should include clear function names and comments for key logic.
Scalability	NFR-SC01	The architecture should allow future integration with larger anonymized healthcare datasets.
Reliability	NFR-R01	App should handle invalid inputs gracefully without crashing.
Ethics	NFR-E01	System shall include disclaimers stating that predictions are not medical diagnoses.

Table 9: Non-functional Requirements

4 System Analysis and Design

4.1 Problem Statement and Software Architecture

4.1.1 Problem Statement

Healthcare teams need effective ways to identify patients who may have elevated stroke risk based on available demographic, clinical, and lifestyle data. Manual review is difficult to scale, and simple rule-based methods may miss complex patterns between variables. At the same time, predictive tools must be explainable, responsibly presented, and carefully evaluated because incorrect predictions can have serious implications.

- **Data Complexity:** Stroke risk may depend on combinations of age, glucose, BMI, hypertension, heart disease, and lifestyle factors.
- **Class Imbalance:** Positive stroke cases may be limited, making standard accuracy insufficient.
- **Deployment Gap:** Healthcare analytics models need usable interfaces, not only notebooks.

4.1.2 Solution Approach

The proposed solution is an end-to-end predictive analytics platform that converts raw healthcare data into actionable stroke risk estimates. The system will provide reproducible notebooks, a trained classification pipeline saved as a reusable model artifact, a Streamlit dashboard for single-patient risk prediction, basic experiment tracking, and monitoring guidelines for future use.

4.1.3 Software Architecture: Layered Design

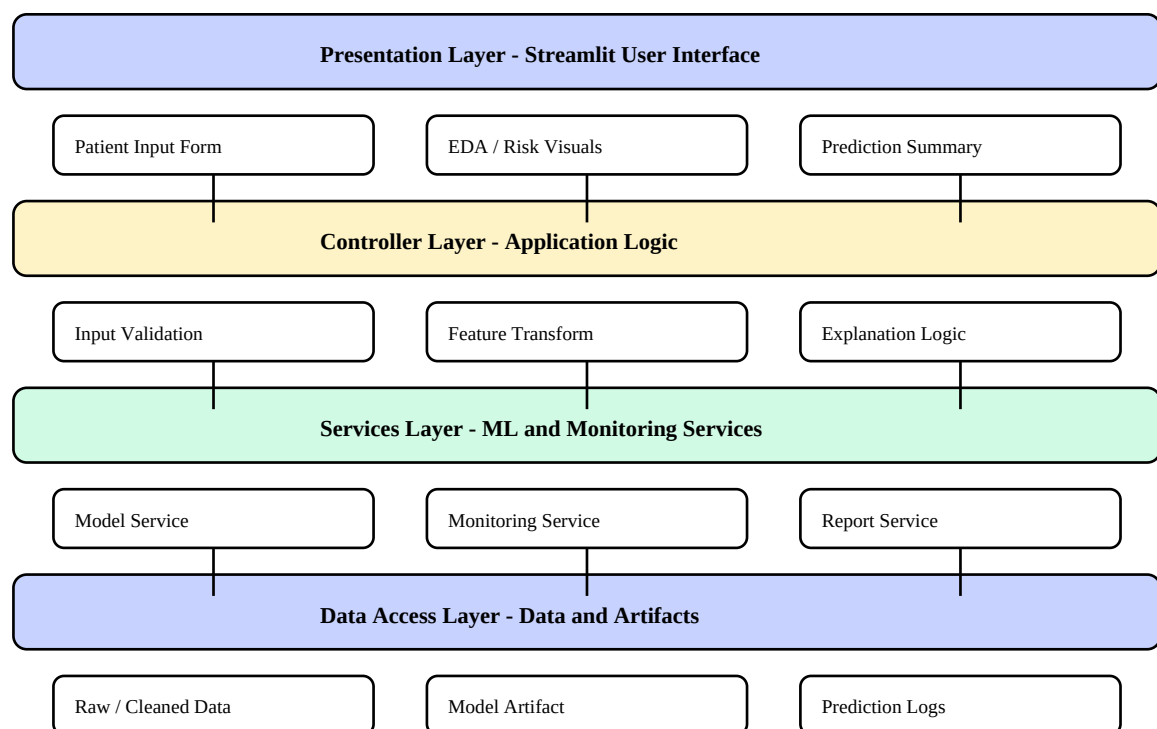


Figure 3: Layered Architecture Diagram for Stroke Risk Prediction System

4.2 Database Design and Data Modeling

The project can be implemented using CSV files and serialized model artifacts during the academic demonstration. For future production use, the following conceptual data model is proposed to store patient inputs, predictions, model versions, and monitoring information.

4.2.1 Conceptual Data Model - Entity Relationship Overview

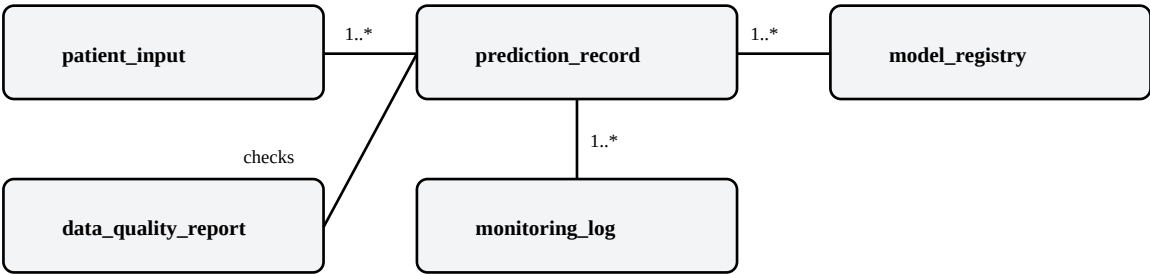


Figure 4: Conceptual Data Model for Healthcare Prediction System

4.2.2 Logical Database Schema

Column Name	Data Type	Constraints	Description
input_id	SERIAL	PRIMARY KEY	Unique record identifier for submitted input.
gender	VARCHAR(20)	NOT NULL	Patient gender category.
age	DECIMAL(5,2)	NOT NULL	Patient age in years.
hypertension	INTEGER	NOT NULL	1 if hypertension is present; 0 otherwise.
heart_disease	INTEGER	NOT NULL	1 if heart disease is present; 0 otherwise.
ever_married	VARCHAR(10)	NOT NULL	Marital status indicator.
work_type	VARCHAR(50)	NOT NULL	Work type category.
residence_type	VARCHAR(50)	NOT NULL	Urban or rural residence category.
avg_glucose_level	DECIMAL(8,2)	NOT NULL	Average glucose level.
bmi	DECIMAL(6,2)	NOT NULL	Body mass index.
smoking_status	VARCHAR(50)	NOT NULL	Smoking behavior category.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Input submission timestamp.

Table 10: Table: patient_input

Column Name	Data Type	Constraints	Description
prediction_id	SERIAL	PRIMARY KEY	Unique prediction record identifier.
input_id	INTEGER	FOREIGN KEY	Link to patient input record.
model_id	INTEGER	FOREIGN KEY	Link to model version used for prediction.
predicted_class	INTEGER	NOT NULL	1 for elevated stroke risk class; 0 otherwise.
risk_probability	DECIMAL(6,5)	NOT NULL	Predicted probability of stroke risk.
threshold_used	DECIMAL(6,5)	NOT NULL	Classification threshold used at prediction time.
risk_label	VARCHAR(50)	NOT NULL	Low, moderate, or high risk label used by app.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Prediction timestamp.

Table 11: Table: prediction_record

Column Name	Data Type	Constraints	Description
model_id	SERIAL	PRIMARY KEY	Unique model version identifier.
model_name	VARCHAR(100)	NOT NULL	Algorithm or pipeline name.
model_version	VARCHAR(30)	NOT NULL	Version identifier such as v1.0.
artifact_path	TEXT	NOT NULL	Path to serialized model pipeline.
roc_auc	DECIMAL(6,5)		Test ROC-AUC score.
recall	DECIMAL(6,5)		Test recall score.
precision_score	DECIMAL(6,5)		Test precision score.
f1_score	DECIMAL(6,5)		Test F1-score.
training_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time model was trained.
status	VARCHAR(20)	NOT NULL	Candidate, production, archived.

Table 12: Table: model_registry

Column Name	Data Type	Constraints	Description
log_id	SERIAL	PRIMARY KEY	Unique monitoring log identifier.
prediction_id	INTEGER	FOREIGN KEY	Related prediction record.
feature_drift_flag	BOOLEAN	DEFAULT FALSE	Indicates whether input drift was detected.
prediction_drift_flag	BOOLEAN	DEFAULT FALSE	Indicates shift in prediction distribution.
validation_status	VARCHAR(30)	NOT NULL	Passed, warning, failed.
message	TEXT		Monitoring details or alerts.
logged_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Monitoring timestamp.

Table 13: Table: monitoring_log

4.2.3 Data Flow Diagram - Level 0 Context Diagram

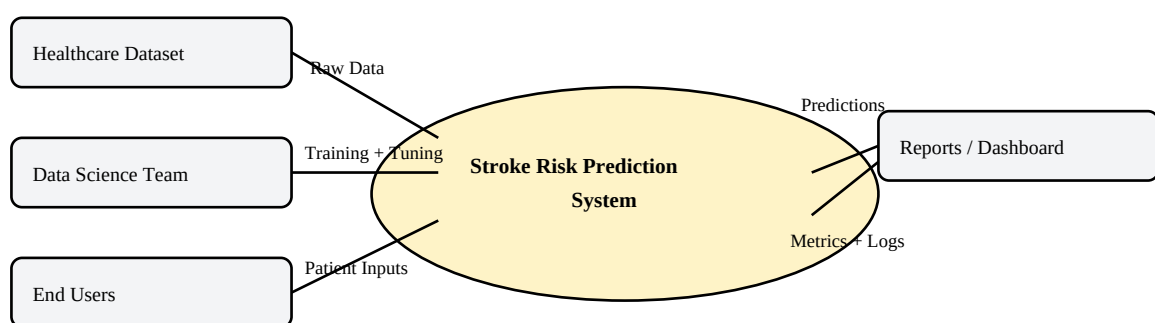


Figure 5: Context Diagram (DFD Level 0)

4.2.4 Wireframe - Dashboard Main View

Section	Content	Purpose
Header	Project title: Healthcare Predictive Analytics / Stroke Risk Prediction; team name; last model update timestamp.	Branding and status visibility.

Section	Content	Purpose
Sidebar Input Panel	Gender, age, hypertension, heart disease, marital status, work type, residence type, glucose level, BMI, smoking status.	Enables real-time patient risk prediction.
Main KPI Row	Predicted class, risk probability, selected threshold, risk category.	Immediate summary of model output.
Risk Explanation Panel	Top contributing features, feature importance chart, patient input summary.	Helps stakeholders interpret the prediction.
EDA Section	Distribution charts, class balance, feature-target charts.	Allows users to understand dataset patterns.
Model Performance Section	Confusion matrix, ROC curve, classification report, model comparison table.	Communicates evaluation evidence.
Footer	Disclaimer, data source note, generated-by text, model limitations.	Responsible use and transparency.

Table 14: Dashboard Wireframe - Component Layout

4.2.5 UI/UX Design Guidelines

- Visual Hierarchy: Prediction result and risk probability appear at the top of the screen.
- Consistent Typography: Clear headings, readable body text, and concise chart labels.
- Color Palette: Navy blue for headers, teal for action buttons, coral red for high-risk warnings, and neutral gray for background sections.
- Input Safety: Numeric inputs have realistic min/max limits; categorical inputs use dropdowns.
- Accessibility: Charts include descriptive titles, text alternatives where possible, and strong color contrast.
- Minimalist Presentation: Avoid unnecessary charts on the prediction page; advanced details appear in expandable sections.
- Error States: App shows clear messages for missing model files, invalid input, or failed prediction.
- Responsible Output: The app clearly states that predictions are analytical estimates and not medical diagnoses.

4.3 Missing Diagrams - Recommendations and Resources

- Sequence Diagram: Shows the prediction request flow from Streamlit input to preprocessing pipeline to model output.
- Activity Diagram: Models the notebook pipeline from data loading through cleaning, EDA, feature engineering, model training, and export.
- State Diagram: Shows model lifecycle states such as candidate, validated, deployed, monitored, retraining required, and archived.
- Class Diagram: Documents implementation classes such as DataProcessor, ModelTrainer, Evaluator, PredictionService, and DriftMonitor.
- Deployment Diagram: Maps the local or cloud deployment environment, including Streamlit app, model artifact, data files, and user browser.
- Component Diagram: Shows dependencies between notebooks, app, serialized pipeline, reports, and monitoring scripts.

5 Expected Final Deliverables

Deliverable	Description
EDA Notebook	Jupyter notebook containing data loading, quality checks, summary statistics, and exploratory visualizations.
Preprocessing Notebook	Notebook documenting imputation, encoding, scaling, duplicate removal, and cleaned dataset export.
Feature Engineering Notebook	Notebook containing engineered features, statistical analysis, feature selection, and feature documentation.
Modeling Notebook	Notebook comparing multiple machine learning models, tuning hyperparameters, evaluating metrics, and selecting the final model.
MLOps Notebook	Notebook or script documenting artifact tracking, model versioning, deployment checks, monitoring strategy, and retraining plan.
Cleaned Dataset	Dataset prepared for modeling and saved separately from the raw input file.
Final Model Artifact	Serialized preprocessing and model pipeline ready for deployment.
Streamlit Application	Python app enabling real-time stroke risk prediction through an interactive interface.
Final Project Report	Comprehensive report covering project planning, data analysis, model development, deployment, monitoring, limitations, and future work.
Final Presentation	Stakeholder-ready presentation explaining methodology, results, app demo, and healthcare value.

Table 15: Final Deliverables Summary

Conclusion

The Healthcare Predictive Analytics / Stroke Risk Prediction project provides a structured end-to-end data science solution for estimating stroke risk from patient demographic, clinical, and lifestyle attributes. The project aligns with the required lifecycle of data collection, preprocessing, analysis, feature engineering, model development, deployment, monitoring, and final documentation.

The proposed solution is valuable because it combines predictive modeling with interpretability, responsible-use communication, and a practical Streamlit deployment. The final model will support educational analytics and risk-awareness workflows, while the documentation will clearly state that predictions are not a substitute for clinical diagnosis. With appropriate future validation, larger datasets, and clinical governance, the system can evolve into a more robust decision-support prototype.

References

- IBM Data Science Projects - Round 4, AI & Data Science Track, Project 3: Healthcare Predictive Analytics Project.
- Scikit-learn documentation for preprocessing pipelines, classification models, model evaluation metrics, and cross-validation.
- Streamlit documentation for building interactive data applications and model demonstration dashboards.
- General healthcare predictive analytics practices for responsible model evaluation, class imbalance handling, and model interpretability.